1328. Break a Palindrome

Medium | Topics | Companies | Hint

Given a palindromic string of lowercase English letters `palindrome`, replace **exactly one** character with any lowercase English letter so that the resulting string is **not** a palindrome and that it is the **lexicographically smallest** one possible.

Return *the resulting string. If there is no way to replace a character to make it not a palindrome, return an* ***empty string****.*

A string `a` is lexicographically smaller than a string `b` (of the same length) if in the first position where `a` and `b` differ, `a` has a character strictly smaller than the corresponding character in `b`. For example, `"abcc"` is lexicographically smaller than `"abcd"` because the first position they differ is at the fourth character, and `'c'` is smaller than `'d'`.

Example 1: Original C++

> **Input:** palindrome = "abccba"
> **Output:** "aaccba"
> **Explanation:** There are many ways to make "abccba" not a palindrome, such as "zbccba", "aaccba", and "abacba".
> Of all the ways, "aaccba" is the lexicographically smallest.

Example 2:

> **Input:** palindrome = "a"
> **Output:** ""
> **Explanation:** There is no way to replace a single character to make "a" not a palindrome, so return an empty string.

Constraints:

- `1 <= palindrome.length <= 1000`
- `palindrome` consists of only lowercase English letters.

AS / Day - 96 / 12 - 11 - 26

leetcode - 1328 - Break a Palindrome

You are given a string called palindrome, the string is guaranteed to be a palindrome.

You have to:

- Replace exactly one char with any lowercase Eng letter
- the resulting string must NOT be a palindrome
- Among all possible valid resulting strings, return the lexicographically smallest one
- If it is impossible, return an empty string "".

What does lexicographically smallest mean?

It means dictionary order.

for ex
- "aabc" is smaller than "abac".
- "ahc" is " " "abd".
- 'a' " " " 'b'.

We compare two 2 strings from left to right, at the 1st position where they differ, the string with the smaller char is lexicographically smaller.

example 1 :- ip input :- palindrome = "abccba"

op "aaccba"

explanation → original string,

a b cc ba

we place the 2nd char b with a,

Now, aa cc ba

- exactly one char changed
- the resulting string is not a palindrome
- it is the lexicographically smallest valid answer.

ex-2 → palindrome : "a"

o/p = " "

why? a single-char string is always a palindrome, no matter which character we replace it with, the resulting string will still be a single-char palindrome, therefore, returning an empty string is necessary.

What should we try to do?

we should try to make the earliest possible character smaller.

since 'a' is the smallest lowercase Eng char, replacing a char with 'a' is our best choice whenever possible.

but we also break the palindrome, this leads us to the greedy approach.

The main greedy obs:-
we need to remember two things:-
1. we want the lexicographically smallest string
2. we need to break the palindrome.

Rule 1 check only the 1st half of the string.
why?
bcz in a palindrome, the 2nd half is just a mirror image of the 1st half. if we change a character in the 1st half, its matching char. in the 2nd half remains unchanged.
Therefore, the palindrome is automatically broken.

Rule 2 → replace the 1st char that is not 'a' with 'a', why?
bcz 'a' is the smallest possible lowercase character.

Also, we choose the earliest possible position to make the new string lexicographically smaller.

Rule 3 → what if every character in the first half is 'a'?

→ then we cannot make any character in the 1st half smaller. in that situation, replace the last character with 'b',

why 'b'?

'a' is the smallest char, but replacing the last char with 'a' would not change it if the last char is already 'a'.

'b' is the smallest character diff from 'a'.

changing the last character breaks the palindrome.

the complete greedy algo:-

1- calculate the length of the string
2- if the length is 1, return "".
3- traverse only the 1st half of the string.
4- find the 1st char that is not 'a'.
5- replace it with 'a' and immediately return the answer
6- if no such char exists, replace the last char with 'b'.
7- return the modified string.

class Solution { public:
string breakPalindrome(string palindrome) {
palindrome = "abccba";
n = 6;
int n = palindrome.length();
if there is only one char, return an empty string.
if (n == 1)
return "";
we traverse only half of the string i.e. n/2 = 6/2 = 3, i=0, i=1, i=2.
for (int i = 0; i < n/2; i++) {
we check where the current char is diff from 'a', but if the current char is already 'a', changing it to another char could make the string lexicographically larger.
if (palindrome[i] != 'a') {
palindrome[i] = 'a'; => replace the 1st char that is not 'a' with a.
return palindrome; } }
we immediately return the result,
but we have already found the earliest position where we can make the string smaller, changing a later position cannot produce a lexicographically smaller answer.
palindrome[n-1] = 'b';
return palindrome;
} this line executes only if every char in the 1st half was already 'a', we replace the last char with 'b', but changing the last char to 'b' breaks the palindrome and keeping all earlier chars unchanged.
finally return the modified string.
};
t.c = O(n)
s.c = O(1)

dry run ex-1 palindrome = "abccba";
length n = 6,
c < 3

iter-1 i = 0,
palindrome[0] = 'a';
curr char is a,
skip it.

iter-2 i = 1;
palindrome[1] = 'b';
'b' != 'a'
replace, aaccba.
↓
required ans.

→ the resulting string is not a palindrome.
→ and we changed exactly one char.

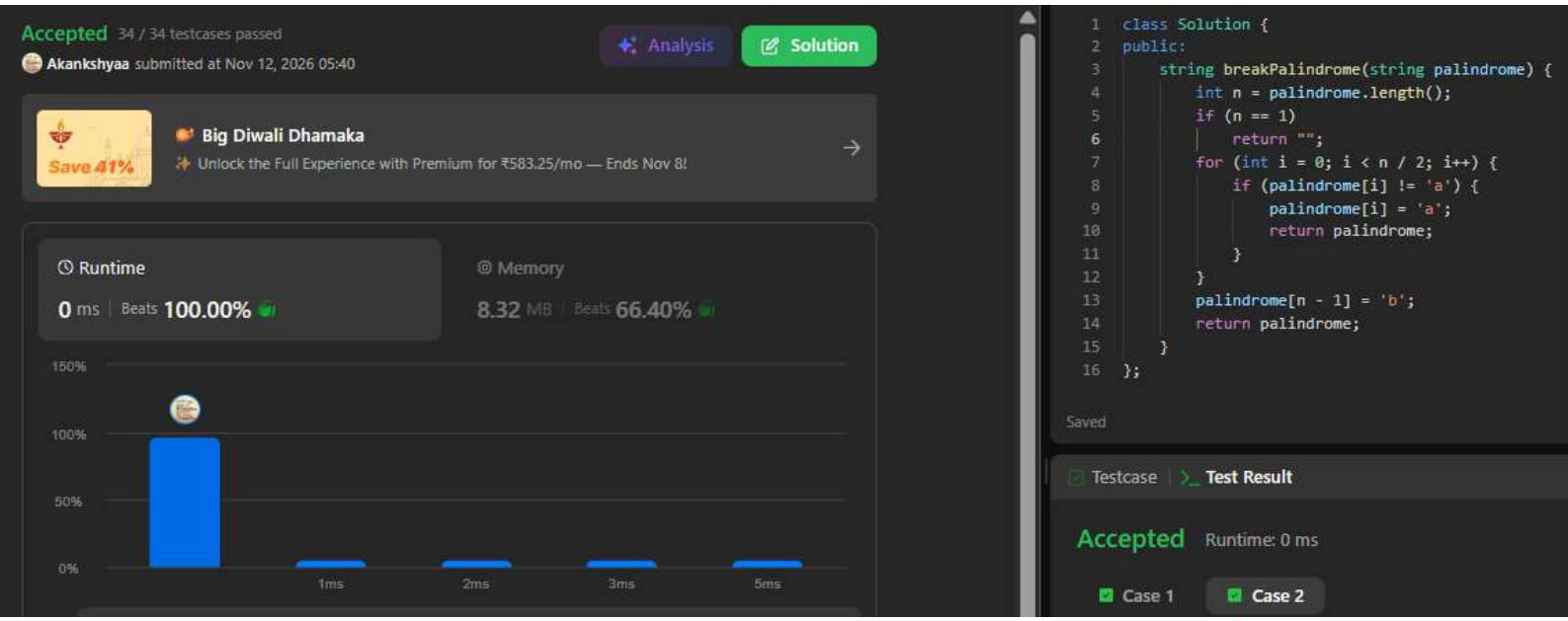
ex-2 palindrome = "aaaa"; n = 4,
i < 4/2, i < 2,

index	char at each	is it diff from 'a'?
0	a	no
1	a	no

the loop finishes without finding any char that can be replaced with 'a',
since n-1 = 3 ⓪, we replace the last char.

a a a a → a a a b
↓
O/P.

→ no longer a palindrome bcz the last char is a, while the last char is b,
→ it is lexicographically the smallest valid answer.




```cpp
class Solution {
public:
    string breakPalindrome(string palindrome) {
        int n = palindrome.length();
        if (n == 1)
            return "";
        for (int i = 0; i < n / 2; i++) {
            if (palindrome[i] != 'a') {
                palindrome[i] = 'a';
                return palindrome;
            }
        }
        palindrome[n - 1] = 'b';
        return palindrome;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2